Frequent Itemset Mining: Framework Design and Performance Analysis of Apriori, PCY, and SON Algorithms

Algorithms for Massive Data – End-Of-Course Project

Leila Cielok

Data Science for Economics (Master's Degree)
II Year

June 29, 2026

I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work, and including any code produced using generative AI systems. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study.

1 Introduction

Frequent Itemset Mining (FIM) and Association Rule Mining (ARM) represent fundamental tasks in data mining and exploratory data analysis. Originally formalized for market basket analysis to discover relationships between products purchased together by customers, these techniques have expanded significantly into diverse domains, allowing this project to apply them to cinematic collaboration networks.

From a computational perspective, finding frequent itemsets presents a major challenge due to the combinatorial explosion of candidate generation. If a dataset contains d unique items, the total search space of potential itemsets spans an unmanageable 2^d combinations. When the support threshold is lowered to capture subtle, high-order correlations, standard counting approaches immediately bottleneck due to CPU or memory limitations.

To address these scalability issues, various algorithmic paradigms have been developed, each introducing unique trade-offs between memory footprints, data passes, and computational overhead. This project evaluates and contrasts three cornerstone methodologies within a unified Python framework:

- Apriori: A candidate generation algorithm that systematically prunes the search space by leveraging the monotonicity principle (an itemset can only be frequent if all of its subsets are also frequent).
- PCY (Park-Chen-Yu): An optimization of the Apriori algorithm that utilizes a localized hashing mechanism and a memory-efficient bit-vector during the first pass to dramatically reduce the candidate pair search space in the second pass.
- SON (Savasere, Omiecinski, and Navathe): A partitioning-based framework designed for large or distributed datasets. It partitions the dataset into independent chunks to find local frequent itemsets, which are subsequently validated in a single global second pass.

By evaluating these implementations on a structured dataset under varying support thresholds and hyperparameter settings, this report characterizes the empirical performance boundaries and efficiency profiles of each approach.

2 Dataset and Pre-Processing Techniques

The empirical analysis is conducted on the IMDb Movies Dataset, which comprises the 1,000 top-rated movies and television series compiled from the official IMDb platform.[1]

The chosen dataset represents a transactional market basket model where each record (basket) maps to a unique movie production, and the individual items within each basket correspond to the cast members (actors) credited in that film.

To eliminate heavy string comparison workloads during processing phases, the framework translates transactional records into integers using an indexing schema:

1. Global Translation Map (`names2int`): A lookup table mapping string actor identifiers to unique integer indexes.
2. Frequent Singleton Array (`frequent_items_table`): An index translation vector constructed after filtering size-1 itemsets. If an item i is determined to be frequent, it receives a secondary compressed identifier ($j \neq 0$); if it fails the threshold, it is explicitly mapped to 0.

[1]The dataset is publicly available on Kaggle: `https://www.kaggle.com/datasets/harshitshankhdhar/imdb-dataset-of-top-1000-movies-and-tv-shows`

3. Reverse Translation Map (`newint2name`): A dynamic map used exclusively at the conclusion of execution loops to re-translate processed numerical itemset pairs or triples back to readable text strings for visual presentation.

3 Algorithms and Implementations

The framework encapsulates three prominent data mining methodologies exposed via clean wrapper interfaces.

3.1 Apriori Implementation

The standard `apriori` implementation proceeds level-by-level. Pairs are formed directly by computing combinations over filtered item vectors. To transition from pairs to triples, the `find_frequent_triples` subroutine evaluates combinations of size-3. It utilizes an optimization trick by casting the validated pair keys into a flat Python set, enabling constant-time $O(1)$ lookups to guarantee that a trio $\{A, B, C\}$ is only tracked if all sub-pairs ($\{A, B\}$, $\{B, C\}$, $\{A, C\}$) exist in the set.

3.2 PCY (Park-Chen-Yu) Implementation

The `pcy` variant optimizes pass 2 by managing an array of integer bucket tracking structures (`bucket_counts`) of size n_{buckets}. During the first pass, as single items are counted, all pairs occurring in a transaction are evaluated via a hash model:

$$\text{hash\_value} = (\text{sorted}(\text{item}_1) \cdot 31) + \text{sorted}(\text{item}_2) \tag{1}$$

$$\text{bucket} = \text{hash\_value} \pmod{n_{\text{buckets}}} \tag{2}$$

A bit-vector (boolean bitmap) is derived by checking whether each bucket value meets the required support. In pass 2, candidate pairs are evaluated only if their assigned hash destination maps to a `True` bit state, skipping non-frequent tracking structures.

3.3 SON Implementation

The `son` framework partitions the global basket array into m equally dimensioned slices (`n_chunks`). It scales the local support threshold dynamically based on chunk fraction metrics:

$$\text{local\_support} = \left\lceil \frac{\text{support\_count} \cdot \text{len(chunk)}}{\text{len(baskets)}} \right\rceil \tag{3}$$

Pass 1 executes the local `apriori` pipeline on each slice, consolidating discovered local frequent patterns into a global candidate structure using union updates (`.update()`). Pass 2 performs a global streaming count over the full dataset to filter out any local false positives.

4 Experimental Methodology and Complexity Scaling

4.1 Workload Complexity Scaling via Support Manipulation

Because the dataset size is fixed at 1,000 rows by external constraints, scalability testing is conducted by manipulating the workload complexity. This is achieved by systematically reducing the support ratio across four test tiers:

$$R = \{0.05,\ 0.01,\ 0.005,\ 0.002\} \tag{4}$$

Lowering the support ratio broadens the search criteria, resulting in an exponential increase in candidate items, pairs, and triples. This tests how each algorithm copes under sudden, combinatorial stress.

4.2 Hyperparameter Optimization Framework

To observe execution dependencies on internal constants, we isolate and benchmark hyperparameter adjustments across standalone loops:

- **PCY Hashing Resolution:** Evaluated over $n_{\text{buckets}} \in \{20, 100, 500, 1000, 5000\}$ to observe collision penalties.
- **SON Chunking Granularity:** Evaluated over $m \in \{4, 10, 20\}$ chunks to examine the trade-offs of localized parallel scaling.

To minimize environmental CPU noise in the execution space, each pipeline iteration is re-run across multiple identical cycles (`N_RUNS`), with performance metrics derived from calculated temporal means.

5 Experimental Results

5.1 Scalability Metrics Table

The empirical properties observed during execution are detailed in the compiled performance registry below.

Table 1: Performance and Itemset Generation Metrics Across Scaling Workloads

Ratio	Count	Apriori (s)	PCY (s)	SON (s)	Singletons	Pairs	Triples	Total
0.050	…	…	…	…	…	…	…	…
0.010	…	…	…	…	…	…	…	…
0.005	…	…	…	…	…	…	…	…
0.002	…	…	…	…	…	…	…	…

5.2 Hyperparameter Inferences

Tables 2 and 3 detail how execution behaviors adapt when tuning internal parameters.

Table 2: PCY Bucket Tuning

Buckets	Time (s)
20	…
100	…
500	…
1000	…
5000	…

Table 3: SON Chunk Tuning

Chunks	Time (s)
4	…
10	…
20	…